STL Commands

STRING

1.) append(): Inserts additional characters at the end of the string. (can also be done using '+' or '+=' operator).
It's time complexity is O(N) where N is the size of the new string.

2.) begin(): Returns an iterator pointing to the first character. Its time complexity is O(1).

3.) clear(): Erases all the contents of the string and assign an empty string ("") of length zero. Its time complexity is O(1).

4.) compare(): Compares the value of the string with the string passed in the parameter and returns an integer accordingly.
Its time complexity is O(N+M) where N is the size of the first string and M is the size of the second string.

5.) copy(): Copies the substring of the string in the string passed as parameter and returns the number of characters copied.
Its time complexity is O(N) where N is the size of the copied string.

6.) empty(): Returns a boolean value, true if the string is empty and false if the

string is not empty. Its time complexity is O(1).

7.) end(): Returns an iterator pointing to a position which is next to the last character. Its time complexity is O(1).

8.) erase(): Deletes a substring of the string. Its time complexity is O(N) where N is the size of the new string.

9.) find(): Searches the string and returns the first occurence of the parameter in the string. Its time complexity is O(N) where N is the size of the string.

10.) insert(): Inserts additional characters into the string at a particular position. Its time complexity is O(N) where N is the size of the new string.

11.) length(): Returns the length of the string. Its time complexity is O(1).

12.) size(): Returns the length of the string. Its time complexity is O(1).

13.) substr(): Returns a string which is the copy of the substring. Its time Complexity is O(N) where N is the size of the substring.

Vector

Vectors are sequence containers that have dynamic size. In other words, vectors are dynamic arrays.
Just like arrays, vector elements are placed in contiguous storage location so they can be accessed and traversed using iterators.
To traverse the vector we need the position of the first and last element in the vector which we can get through begin() and end() or we can use indexing from 0 to size().

Some of the Member Functions of Vectors are:-

at(): Returns the reference to the element at a particular position (can also be done using '[]' operator).
It's time complexity is O(1).

back(): Returns the reference to the last element.
It's time complexity is O(1).

begin(): Returns an iterator pointing to the first element of the

Vector: It's time Complexity is O(1).

clear(): Deletes all the elements from the vector and assign an empty vector.
It's time complexity is O(N) where N is the size of the vector.

empty(): Returns a boolean value, true if the vector is empty and false if the vector is not empty. its time complexity is O(1).

end(): Returns an iterator pointing to a position which is next to the last element of the vector.

erase(): Deletes a single elements or a range of elements. It's time complexity is O(N+M) where N is the number of the elements erased and M is the number of the elements moved.

front(): Returns the reference to the first element. It's time complexity is O(1).

insert(): Inserts new elements into the vector at a particular position. Its time complexity is O(N+M) where N is the number of elements inserted and M is the number of elements moved.

pop_back(): Inserts a new element from the vector. It's time complexity is O(1).

push_back(): Inserts a new elements at the end of the vector. It's time complexity is O(1).

resize(): Resizes the vector to the new length which can be less than or greater than the current length. It's time complexity is O(N) where N is the size of the resized vector.

size(): Returns the number of elements in the vector. It's time complexity is O(1).

List

Page No.
Date

List is a sequence container which takes constant time in inserting and removing elements. List in STL is implemented as Double Link List. The elements from List cannot be directly accessed.

For example:- To access elements of a particular position, you have to iterate from a known position to that particular position.

begin(): It returns an iterator pointing to the first element in List. Its time complexity is O(1).

end(): It returns an iterator referring to the theoretical element (doesn't point to an element) which follows the last element. Its time complexity is O(1).

empty(): It returns whether the list is empty or not. It returns 1 if the list is empty otherwise returns 0. It's time complexity is O(1).

back(): It returns reference to the last element in the list. It's time complexity is O(1).

assign(): It assigns new elements to the list by replacing its current elements and change it's size accordingly. It's time complexity is O(N).

erase(): It removes a single element or the range of element from the list. It's time complexity is O(N).

front(): It returns reference to the first element in the list. It's time complexity is O(1).

push-back(): It adds a new element at the end of the list, after its current last element. It's time complexity is O(1).

remove(): It removes all the elements from the list, which are equal to given element. Its time complexity is O(N).

pop-back(): It removes the last element of the list, thus reducing its size by 1. Its time complexity is O(1).

pop-front(): It removes the first element of the list, thus reducing its size by 1. It's time complexity is O(1).

insert(): It insert new elements in the list before the element on the specified position. It's time complexity is O(N).

reverse( ): It reverses the order of elements in the list. It's time complexity is O(N).

size( ): It returns the number of elements in the list. It's time complexity is O(1).

Pair

Pair is a container that can be used to bind together a two values which may be of different types.
Pair provides a way to store two Heterogeneous objects as a single unit.

```
pair<int, char> p1; // default
pair<int, char> p2 (1, 'a'); //value initialization.
pair<int, char> p3 (p2); //copy of p2.
```

We can also initialize a pair using make_pair() function.
make_pair(x,y) will return a pair with first element set to x and second element set to y.

```
p1 = make_pair(2, 'b');
```

To access the elements we use keywords, first and second to access the first and second element respectively.

```
cout<<p2.first<<" "<<p2.second<<endl;
```

Set and Multiset

Page No.
Date

Sets are containers which store only Unique values and permit easy look ups.

→ The values in the sets are stored in some specific order (like ascending or descending).

→ Elements can only be inserted or deleted, but cannot be modified.

→ We can access and traverse set elements

Multisets :- are containers that store elements following a specific order, and where multiple elements can have equivalent values.

In a multiset, the value of an element also identifies it (the value is itself the key, of type T). The value of the elements in a multiset cannot be modified once in the container (the elements are always constant), but they can be inserted or removed from the container.

```
set<int> s1; //Empty Set
int a[] = {1, 2, 3, 4, 5, 5};
set<int> s2(a, a+6); // s2 = {1, 2, 3, 4, 5}
set<int> s3(s2); // Copy of s2
set<int> s4(s3.begin(), s3.end()); // Set created using iterators.
```

```
multiset<int> first; //empty multiset of ints

int myints[] = {10, 20, 30, 20, 20};
multiset<int> second (myints, myints+5); //pointers
multiset<int> third (second); a copy of second
multiset<int> fourth(second.begin(), second.end());
   //multiset created using iterators.
```

Member functions of Set are:

begin(): Returns an iterator to the first element of the set. It's time complexity is O(1).

clear(): Deletes all the elements in the set and the set will be empty. Its time complexity is O(N) where N is the size of the set.

count(): Returns 1 or 0 if the element is in the set or not respectively. Its time complexity is O(logN) where N is the size of the set.

empty(): Returns true if the set is empty and false if the set has at least one element. Its time complexity is O(1).

end(): Returns an iterator pointing to a position which is next to the last element. Its time complexity is O(1).

Page No.
Date

Erase(): Deletes a particular element or a range of elements from the set. Its time complexity is O(N) where N is the number of element deleted.

Find(): Searches for a particular element and returns the iterator pointing to the element if the element is found otherwise it will return the iterator returned by end(). Its time complexity is O(log N) where N is the size of the set.

Insert(): insert a new element. Its time complexity is O(log N) where N is the size of the set.

Size(): Returns the size of the set or the number of elements in the set. Its time complexity is O(1).

Maps

Maps are containers which store elements by mapping their value against a particular key. It stores the combination of key value and mapped value following a specific order. Here key value are used to uniquely identify the elements mapped to it.

The data type of key value and mapped value can be different. Elements in map are always in sorted order by their corresponding key and can be accessed directly by their key using bracket operator ([]).

In map, key and mapped value have a pair type combination, i.e. both key and mapped value can be accessed using pair type functionalities with the help of iterators.

```
map<char, int> mp;
mp['b'] = 1;
```

1.) at(): Returns a reference to the mapped value of the element identified with key. It's time complexity is O(log n).

2.) count(): Searches the map for the elements mapped by the given key and returns the number of matches. As map stores each element with unique key, then it will return 1 if match if found otherwise return 0.

TC: O(log N).

3.) clear(): Clears the map, by removing all the elements from the map and leaving it with its size 0.
TC: O(N).

4.) begin(): Returns an iterator referring to the first element of map. Its time complexity is O(N).

5.) end(): Returns an iterator referring to the theoretical element (doesn't point to an element) which follows the last element.
TC: O(1).

6.) empty(): Checks whether the map is empty or not. It doesn't modify the map. It returns 1 if the map is empty otherwise returns 0.
TC: O(1).

7.) erase(): Removes a single element or the range of element from the map.

8.) find(): Searches the map for the element with the given key, and returns an iterator to it, if it is present in the map otherwise it returns an iterator to the theoretical element which follows the last element of map.
TC: O(log N).

9.) Insert(): Insert a singk element or the range of element in the map.
TC = O(logN), when only element is inserted, and O(1) when position is also given.

Unordered Maps

NOTE:- Maps and UnOrdered Maps have almost same functions, but have different underlying implementations. We say Unordered maps take O(1) time for search, insert and erase in average case, hence are very Helpful.

1.) find(): Searches the container for an element with K as key and returns an iterator to it if found, otherwise it returns an iterator to unordered_map::end.

2.) rehash(): Sets the number of buckets in the container to n.

3.) insert(): inserts a new key-value pair into the container.

4.) erase(): Removes from unordered_map container either a single element or a range of elements.

5.) count(): This function returns 1 if an element with that key exists in the container, and zero otherwise.

6.) load_factor(): this function returns a floating value denoting current load factor in the unordered_map container.
load_factor = current_size/bucket_count

7.) clear(): Clears the map, by removing all the elements from the map and leaving it with its size 0.
T.C. = O(N).

8.) begin(): Returns an iterator referring to the first element of the map.
TC = O(1).

9.) end(): Returns an iterator referring to the theoritical element (doesn't point to an element) which follows the last element.
TC = O(1).

10.) Operator[]: If k matches the key of an element in the container, the function returns a reference to its mapped value.

Stack

Page No.
Date

Stack is a container which follows the LIFO (Last in First Out) order and the elements are inserted and deleted from one end of the container. The element which is inserted last will be extracted first.

```
stack<int>s;
```

1.) Push(): Insert element at the TOP of stack.
TC = O(1).

2.) POP(): Removes element from top of stack.
TC = O(1).

3.) top(): Access the top element of stack.
TC = O(1).

4.) empty(): checks if the stack is empty or not.
TC = O(1).

5.) size(): Returns the size of stack.
TC = O(1).

Queue

Queue is a container which follows FIFO. Elements inserted at rear and extracted from front.

```
queue <int> q;
```

push(): inserts an element in queue at one end (rear).
TC = O(1)

pop(): Deletes an element from another end if queue (front).
TC = O(1)

front(): Access the element on the front end of queue.
TC = O(1)

empty(): Checks if the queue is empty or not.
TC = O(1)

size(): Returns the size of queue.
TC = O(1).

Priority Queue

A priority queue is a container that provides constant time extraction of the largest element, at the expense of logarithmic insertion. It is similar to the heap in which we can add element at any time but only the maximum element can be retrieved.

In a priority queue, an element with high priority is served before an element with low priority.

```
priority_queue<int> pq;
```

To make a min-priority queue, declare priority queue as:

```
#include<functional> //for greater<int>
//min priority queue.
priority_queue<int, vector<int>, greater<int>
> pq;
```

empty(): Returns true if the priority queue is empty and false if the priority queue has at least one element.
TC = O(1).

pop(): Removes the largest element from the priority queue.
TC = O(logN) where N is the size of the priority queue.

push(): Inserts a new element in the priority queue. TC = O(logN) where N is the size of the priority queue.

size(): Returns the number of element in the priority queue.
TC = O(1).

Top(): Returns a reference to the largest element in the priority queue.
TC = O(1).

DeQue

Page No.
Date

Double-ended queues are sequence containers with dynamic sizes that can be expanded or contracted on both ends (either its front or its back).

```
deque<int> first; //empty deque of integer
deque<int> second (4,100); //four ints with value 100
deque<int> third (second.begin(), second.end()); //iterating
deque<int> fourth (third); //a copy of third
```

1. assign(): Assigns new contents to the deque container, replacing its current contents, and modifying its size accordingly.

2. at(n): Returns a reference to the element at position n in the deque container object.

3. back(): Returns a reference to the last element in the container.

4) begin(): Returns an iterator pointing to the first element in the deque container.

5) empty(): Returns whether the deque container is empty. (i.e. whether its size is 0).

6) end(): Returns an iterator referring to the past-the-end element in the deque container.

(7.) erase(): Removes from the deque container either a single element (position) or a range of elements ~~([first]~~ ([first, last]).

(8.) front(): Returns a reference to the first element in the deque container.

100 Iterating

(9.) pop_back(): Removes the last element in the deque container, effectively reducing the container size by one.

(10.) pop_front(): Removes the first element in the deque container, effectively reducing its size by one.

(11.) push_back(): Adds a new element at the end of the deque container, after its current last element.

(12.) push_front(): Inserts a new element at the beginner of the deque container, right before its current first element.

(13.) size(): Returns the number of elements in the deque container.

Iterator

An iterator is any object that, points to some element in a range of elements (such as an array or a container) and has the ability to iterate through those elements using a set of operators (with at least the increament (++) and dereference (*) operators).

For Vector:

```
vector<int> :: iterator it;
```

For List:

```
list<int> :: iterator it;
```

etc.....

ALGORITHM

1.) <algorithm> = The header <algorithm> defines a collection of functions especially designed to be used on ranges of elements.

2.) binary_search (first, last, val)
Returns true if any element in the range [first, last) is equivalent to val, and false otherwise.

```
binary_search (v.begin(), v.end(), 3))
    //v is a vector.
```

3.) find(first, last, val)
Returns an iterator to the first element in the range [first, last) that compares equal to val. If no such element is found, the function returns last.

```
it = find (myvector.begin(), myvector.end(), 30);
    //it is an iterator.
```

4.) lower_bound(first, second, val)
Returns an iterator pointing to the first element in the range [first, last) which does not compare less than val.

```
it = lower_bound (v.begin(), v.end(), 20);
    //v is a vector.
```

5.) upper_bound(first, second, val)
Returns an iterator pointing to the first element in the range [first, last) which compares greater than val.

it = upper_bound(v.begin(), v.end(), 20);
// v is a vector.

6.) max(a,b) : Returns the largest of a and b. If both are equivalent, a is returned.
cout << max(a,b);

7.) min(a,b) : Returns the smallest of a and b. If both are equivalent, a is returned.
cout << min(a,b);

8.) reverse(first, last) : Reverses the order of the elements in the range [first, last).
reverse(myvector.begin(), myvector.end());

9.) rotate(first, middle, last)
Rotates the order of the elements in the range [first, last) in such a way that the element pointed by middle becomes the new first element.
rotate(myvector.begin(), myvector.begin()+3, myvector.end());

10.) sort(first, last)
Sorts the elements in the range [first, last) into ascending order.
sort(v.begin(), v.end());
sort(a, a+n);
sort(v.begin(), v.end(), comparator);
sort(a, a+n, comparator);

11.) swap(a,b) : Exchanges the values of a and b.

```
swap(a,b);
```

12.) next_permutation(first, last)

Rearranges the elements in the range [first, last) into the next lexiographically greater permutation.

```
next_permutation(v.begin(), v.end());
```